INL/RPT-25-87095

3.1.3 Attack Methodology and Framework

Patience Yockey

Jeremy Jones

Juliana Ocampo Giraldo

Ross Kunz

Bradley Marx

Alex Kaforey

Battelle Energy Alliance manages INL for the
U.S. Department of Energy's Office of Nuclear Energy

Idaho National Laboratory

DISCLAIMER

This information was prepared as an account of work sponsored by an agency of the U.S. Government. Neither the U.S. Government nor any agency thereof, nor any of their employees, makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness, of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. References herein to any specific commercial product, process, or service by trade name, trade mark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the U.S. Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the U.S. Government or any agency thereof.

3.1.3 Attack Methodology and Framework

1. SUMMARY

Pre-existing AI red teaming frameworks have issues with being highly qualitative and large language model (LLM) focused, misaligning with electric grid-oriented tasks and missing other model architectures.

- AMARANTH establishes a model agnostic testing framework to determine how models will behave during deployment through five key steps:
  - Retrieve a high maturity AI model from vendor/industry partner
  - Identify model attack vectors and vulnerabilities
  - Integrate AI model into a testbed to determine the downstream effects of AI
  - Determine resilience and capacity under normal and adversarial conditions
  - Provide report of model resilience and capacity to vendor/industry partner to help inform implementation.
- AMARANTH focuses on model attack vectors as they remain understudied in the red teaming world, where four key attacks against ML systems were identified:
  - Model Inversion: Adversary obtaining proxy model for attack planning and testing
  - Weight Manipulation: change weights and parameters to suit adversary's goal in a certain timespan
  - Architecture Manipulation: change architecture of model (optimizers, types of layers, activation functions, etc.) to suit adversary's goal in a certain timespan
  - Seed Manipulation: seeds to cherry-pick favorable outcomes and fake performance metrics.
- AMARANTH attack scope includes an APT group targeting the electric industry with delayed/gradual action on objectives with desire to cause system failure overtime without significant detection, impacting ML/AI resilience and capacity.
- AMARANTH attack path includes:
  - Model inversion
  - Model access (out of scope as it is a traditional and well-studied attack vector)
  - Model manipulation (weight, architecture, or seed manipulation)

2. PRE-EXISTING AI EVALUATION FRAMEWORKS

There is a multitude of work on adversarial actions against machine learning and artificial intelligence (ML/AI). However, much of the methodology focuses on generative AI (GenAI) and large language models such as MITRE ATLAS (MITRE, n.d.). OWASP maintains a series of machine learning risks such as the Top 10 Machine Learning Security Risk (Singh et al. 2023) but does not have a set framework for red teaming and a limited scope. Microsoft AI red team (Kumar 2023) has a red teaming framework and threat modeling guidance; however, these risks are inherently qualitative in nature. Table 1 shows identified advantages and disadvantages of popular AI red teaming frameworks, showing a lack of quantitative and model agnostic testing frameworks.

Table 1. Benefits quantification of AI red teaming frameworks.

Framework	Advantages	Disadvantages
MITRE ATLAS	Direct tactics, techniques, and procedures (TTP) mapping Developed by a variety of partners Provides case studies Provides mitigations	Focuses mainly on Large Language Models (LLMs) Qualitative
Microsoft AI Red Team	Clear instructions for red teaming Provides an open-source framework for generative AI red teaming Provides mitigations Has questions to ask in an AI/ML security review	Qualitative Largely focused on LLM red teaming
OWASP Top 10 ML Security Risks	Quantifies risk factors for example attack scenarios Unique attack vectors	Not a red teaming framework Limited scope

Quantitative metrics of risk exist with the data side of ML; however, model red teaming remains understudied. OWASP gets close to quantification of exploitability, security weakness, and impact of model and data risks, however the risk is defined 1 through 5 for a given toy problem and through arbitrary or expert solicitation.

As the electric industry moves closer to implementation, resiliency and capacity quantification remains a key concern following deployment (Tabassi 2023) which remains understudied with respect to adversarial model manipulations. Artificial Intelligence Management and Research for Advanced Networked Testbed Hub (AMARANTH) aims to conduct model-focused attacks to determine the resiliency of models under adversarial action, determining if attacks impact the model drift and time to retraining, providing key industry partners recommendations to maintenance intervals and determining cyber-effects.

3. AMARANTH TESTING FRAMEWORK

The AMARANTH testing framework provides an evaluation framework to test high technology readiness level (TRL) models prior to deployment in the electric sector. Through working with vendors and industry partners, AMARANTH determines the most impactful AI models to test to align with the energy sector's goals. AMARANTH's unique focus on model attack vectors and vulnerabilities gives the framework a unique perspective outside of the traditional data security space. The models are then fit into a testbed to analyze the effects of normal AI operation. Using faster than real-time simulations, AMARANTH can determine resilience and capacity through calculating model drift and amount of time it takes for a model to complete a task under normal and adversarial conditions. Finally, a report using quantitative metrics achieved in the previous step is shared with the vendor/industry partner to help inform the resilience of the model to cyberattacks, the capacity of the model in a realistic environment, and the amount of time a model can be kept in deployment before needing to be retrained. The AMARANTH testing framework is exhibited in Figure 1, where the key five steps are enumerated with the goal to make a scalable AI testing framework which is model agnostic.

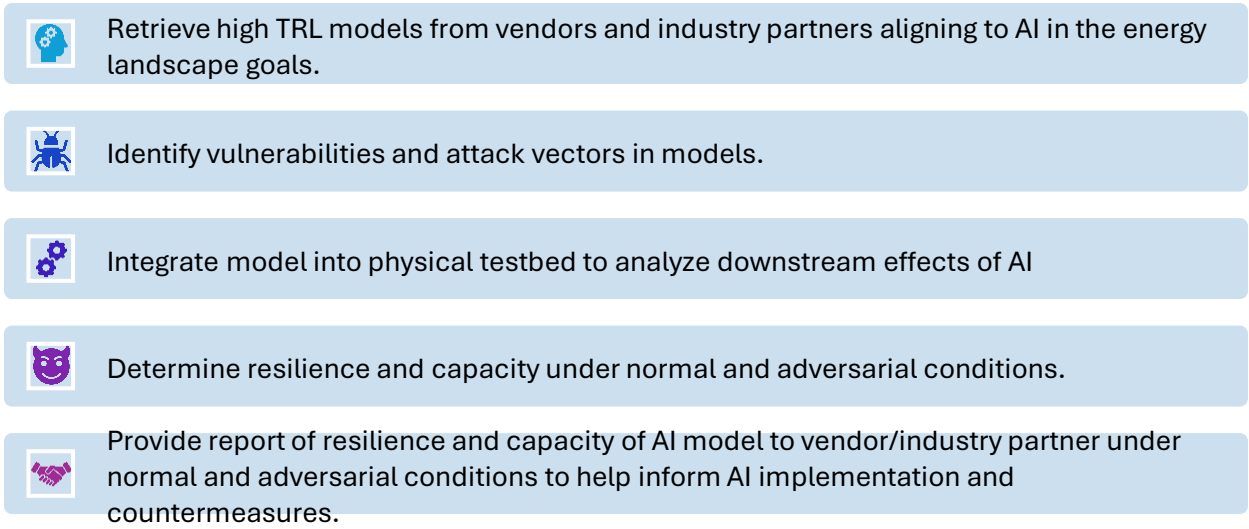

Figure 1. AMARANTH testing process.

4. AMARANTH ATTACK VECTORS

In the AMARANTH project, we assume a model is thoroughly trained and tested and in deployment. Additionally, we assume an adversary has means to get initial access to the model in deployment. As we are focused on exclusively model attack vectors, we consider the following attacks:

1. Model inversion.
2. Architecture manipulation.
3. Weight manipulation.
4. Seed manipulation.

We assume an advanced persistent threat (APT) level of skill and objectives, where a key goal is obfuscation of an attack through delayed or gradual action on objectives which reduces the chance of detection and can make attacks look like legitimate system failures (Liu et al. 2022).In this project, we consider an APT group targeting the electric industry with delayed/gradual action on objectives with desire to cause system failure overtime without significant detection, impacting ML/AI resilience and capacity. As we focus on ML/AI security, the generalized attack path is as follows in Figure 2, ignoring extraneous systems and attack vectors prior to gaining model access.

Figure 2. AMARANTH attack path.

The adversary therefore does model inversion if given a black box system where only outputs are given, accesses the model through model artifacts or the inverted model, manipulates weights, seed, or architecture, and then redeploys changed model to cause impact. In this instance, we do not cover the point up to inversion or how the model is redeployed in an operational space as traditional OT adversarial techniques will suffice.

4.1. Model Inversion

The initial step of the attack path is model inversion. Model inversion is an attack where an adversary uses a model's outputs to recover its architecture. Having recovered the model's architecture, an adversary can construct an attack testbed. An attack testbed enables offline experimentation with various attack vectors. Offline experimentation conceals the adversary's experiments from defense mechanisms.

In our red teaming methodology, we consider two model inversion scenarios:

1. White Box: The model's architecture, files, data, etc., is self-contained on a compromised system where an adversary can exfiltrate the model and dependencies to test-out manipulation techniques.
2. Black Box: Only the model output and basic information (model's purpose and rough estimate of training data) is available. In this case, the adversary must use a surrogate dataset containing roughly the model's original data to iterate on to produce a model and output like the original output.

In this process, we consider the two extremes of model inversion, where most models typically fall in between these two extremes. Models can leave checkpoint data or model save files which can assist with model inversion and architecture understanding, models might have training data available on the system, or they might have executable files which can be used to derive model architecture, weights, etc.

4.2. Model Manipulation

We consider four different types of manipulation of ML/AI models within AMARANTH: Weight manipulation, architecture manipulation, and seed manipulation to determine the effect of model-based attacks on resilience.

4.2.1. Weight Manipulation

Weight manipulation is the process of manipulating parameters of ML/AI algorithms. The idea is simple: iterate through a series of weights to increase or decrease a result based on adversarial goals (Kurita et al. 2020).

For example, different models have different parameters which can be manipulated such as multilayer-perceptron (MLP) neural network parameters (see Figure 3). Each of these functions such as layer sizes (number of neurons in a layer), the alpha ($L2$ regularization term strength), learning rate (schedule for weight updates), and the maximum number of iterations (number of iterations unless the model converges before then) can be adversarially modified to give different results (Rakin et al. 2021).

```
# define potential sets of hyperparameters
hidden_layer_sizes = [(100,),(500,),(100,10,),(100,100,)]
alpha = [0.0001, 0.00001, 0.001]
learning_rate = [0.0001, 0.001, 0.01]
max_iter = [200, 1000, 10000]
grid_search = pd.DataFrame(columns=['hl','a','lr','mi','mae'])
```

Figure 3. Multilayer-perceptron neural network parameters.

4.2.2. Architecture Manipulation

Similar to weight manipulation, architecture manipulation is the manipulation of architecture of the AI/ML model such as choosing different optimizers, different types of layers and activation functions, and different types of loss and metrics (Shkulov 2023).An adversary can manipulate the architecture of the model to allow for the model to change architectures to produce different results which benefit the adversary.

4.2.3. Seed Manipulation

ML/AI requires random numbers generated by a pseudo random number generator to generate the results and generate performance metrics. Through seed manipulation. And adversary can manipulate the seed to cherry-pick favorable outcomes to mask the model's true performance and cause failures in production, leaking information from the test set (Brownlee 2024). Seed manipulation can impact data splitting, resampling, weight initialization, hyperparameter optimization, data augmentation, and feature engineering. Without a robust model, seed manipulations can impact model resilience.

5. REFERENCES

Brownlee, Jason. 2024. "Machine Learning Mischief." GitHub, December 20. https://github.com/Jason2Brownlee/MachineLearningMischief/blob/main/examples/seed_hacking.md.

Kumar, Ram Shankar Siva. 2023. "Microsoft AI Red Team Building Future of Safer AI." *Microsoft Security Blog*, August 7. https://www.microsoft.com/en-us/security/blog/2023/08/07/microsoft-ai-red-team-building-future-of-safer-ai/.

Kurita, Keita, Paul Michel, and Graham Neubig. 2020. "Weight Poisoning Attacks on Pretrained Models." In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, edited by Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault. Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.acl-main.249.

Liu, Jianyi, Ying Liu, Jingwen Li, et al. 2022. "Two Statistical Traffic Features for Certain APT Group Identification." *Journal of Information Security and Applications* 67 (June): 103207. https://doi.org/10.1016/j.jisa.2022.103207.

MITRE. n.d. "ATLAS Matrix." MITRE ATLAS. Accessed July 17, 2025. https://atlas.mitre.org/matrices/ATLAS.

Rakin, Adnan Siraj, Zhezhi He, Jingtao Li, Fan Yao, Chaitali Chakrabarti, and Deliang Fan. 2021. "T-BFA: Targeted Bit-Flip Adversarial Weight Attack." arXiv:2007.12336. Preprint, arXiv, January 8. https://doi.org/10.48550/arXiv.2007.12336.

Shkulov, Valentine. 2023. "Neural Network Layers: All You Need Is Inside Comprehensive Overview | HackerNoon." Hackernoon, July 17. https://hackernoon.com/neural-network-layers-all-you-need-is-an-inside-comprehensive-overview.

Singh, Shain, Sagar Bhure, and Ben Kereopa-Yorke. 2023. "OWASP Machine Learning Security Top Ten." OWASP. https://owasp.org/www-project-machine-learning-security-top-10/.

Tabassi, Elham. 2023. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology. https://doi.org/10.6028/nist.ai.100-1.